IMGD 4000 Portfolio
Ananya Jayamoorthy
Team OOMFs

I. My Role in the Project

I had a few main tasks as a programmer on this project.

- Creating the portals that allowed a player to move from “world” to “world”
- Making a dialogue/narrative system that was scalable with the amount of text we wanted to include on the game
- Play around with Niagara Systems to add to the feel and vibe of the environment (tech art tasks)
    - Glow Cave had atmospheric glow particles
    - Abyss had a particle based waterfall
    - Abyss had a water material that reacted to the player’s footsteps
    - Local volumetric fog was created but not used
- Handle the wall-disappearing feature so that the player’s view is not obstructed by a wall

II. Challenges I Faced

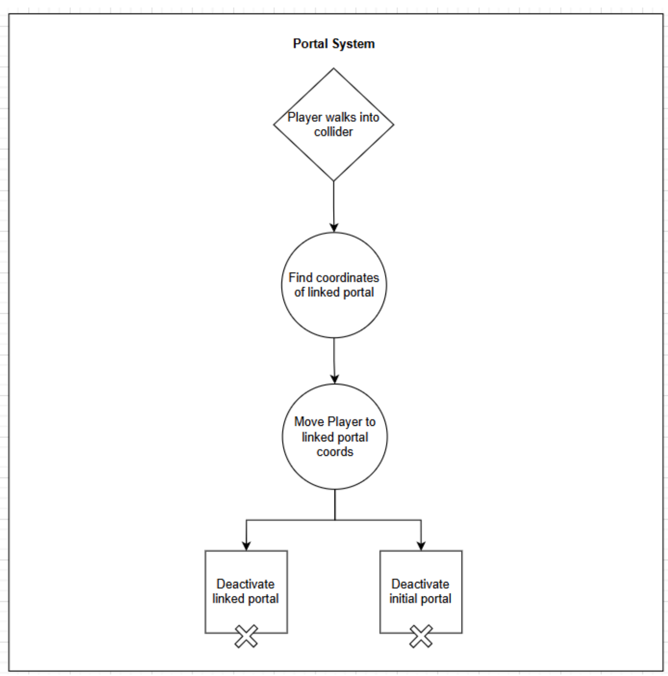
1. Niagara Systems are difficult. There are so many small components that go into building an emitter system and it can really put a strain on your computer. Another issue that I ran into is that a lot of the tutorials that were online for the specific Niagara effects I wanted were made for UE4. These tutorials were in a version of the engine that is VASTLY different from what was used for this project.
2. This was my first time working in Unreal and there is a LOT to keep track of and learn. While this project used blueprints, I found myself wanting to work with C++ because at least that was a language I was familiar with. I have, however, heard that C++ can be a worse development experience. Overall it was an insane learning curve.

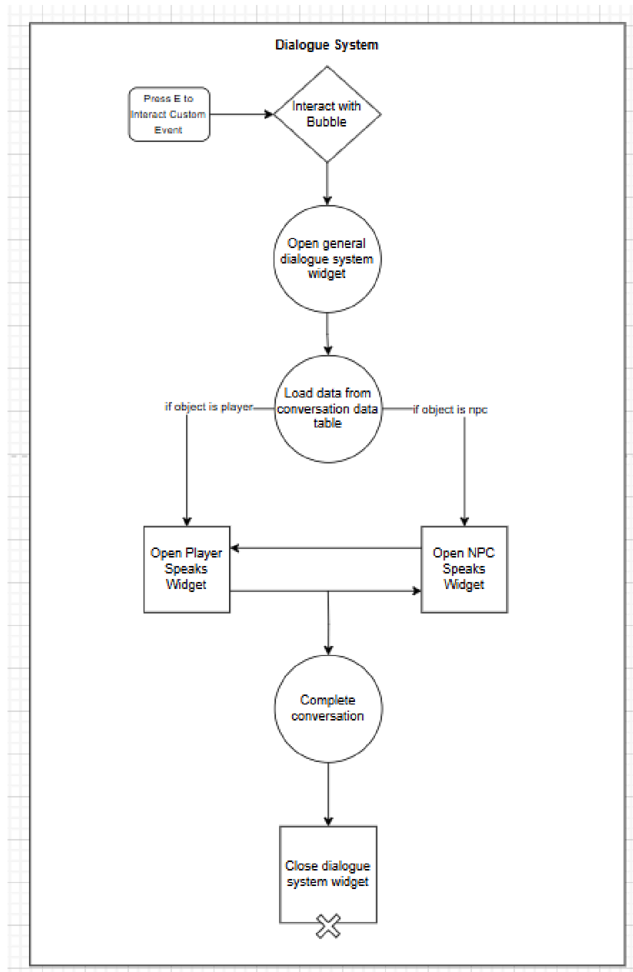
III. How you overcame those challenges, including specific APIs used and code fragments (both Blueprints and C++)

My issues were more general. My biggest resources for dealing with the above challenges were the internet and my tech teammates, both of which had prior Unreal experience. As the features I was implementing weren’t super crazy and difficult, a quick google search lead me to others who had faced (and solved) the same problems I was experiencing. For smaller issues I turned to my teammates who either showed me how to fix my problem or sat with me to help me debug my issues.

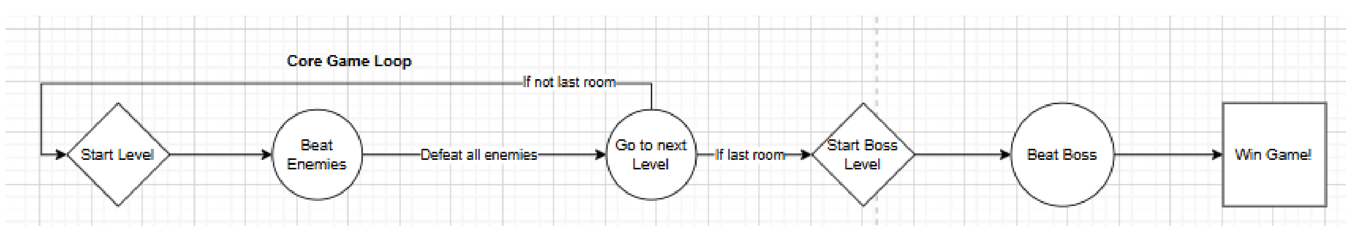
IV. An architectural diagram(s) that roughly portrays the architecture of your final project and then shows the design of your specific parts in more detail. This might include UML, control flow diagram, networking diagrams etc.

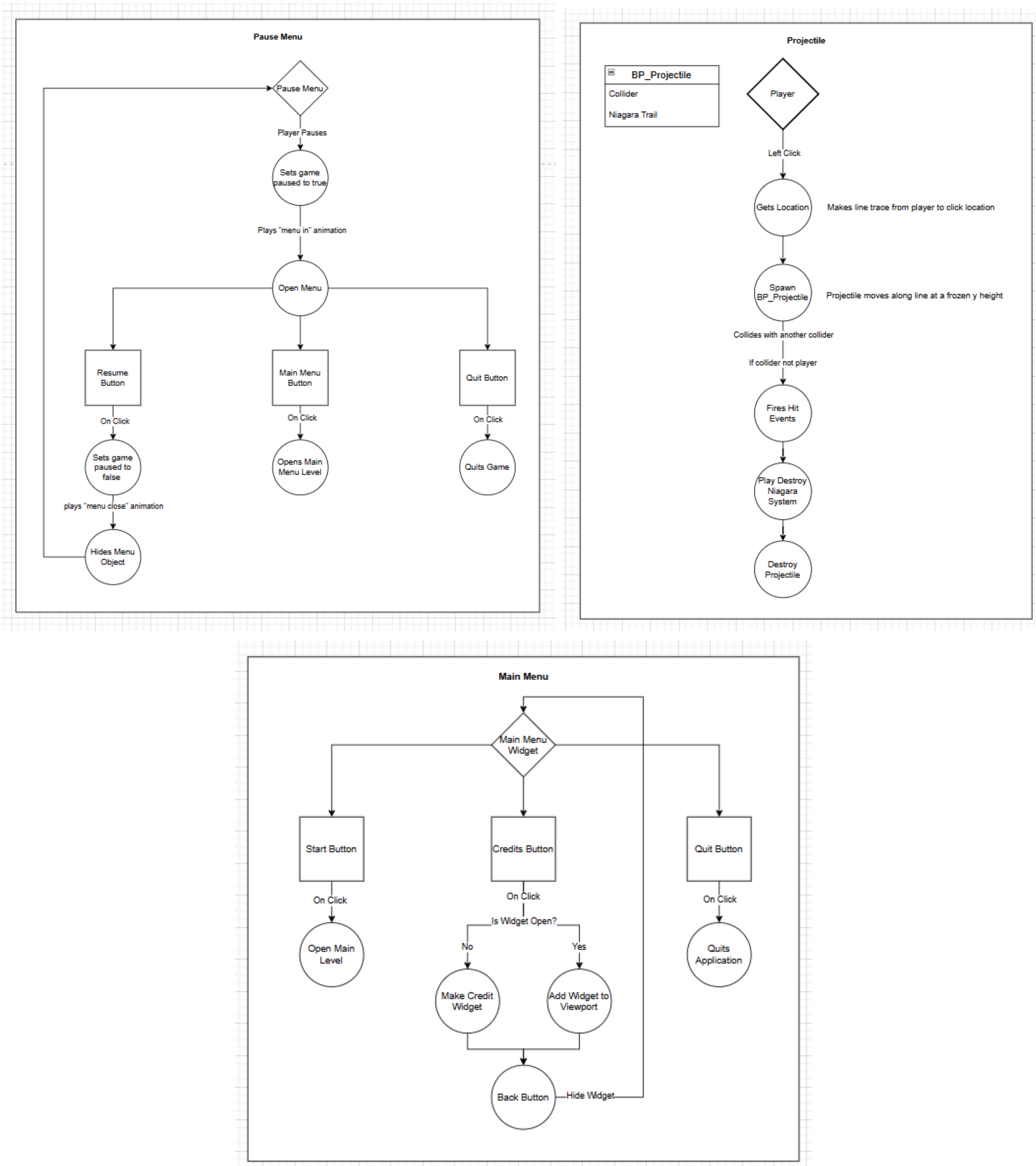
Diagrams for Features I Worked On:





General Game Diagrams:





V. Lessons learned: how should students facing similar challenges in the future tackle the problems you faced?

Two pieces of advice I would give students facing these challenges would be:

1. If you find a tutorial that shows you how to do something you want to implement, watch the tutorial all the way through at least once to see if it is doable. It is not uncommon to come across tutorials that expect a certain level of base knowledge or other skills (such as 3D modeling).
2. Talk to other people! It may seem difficult to talk to people you don't know but everyone has a different skill set and skill level and oftentimes asking a person how to fix something

can be a lot faster than googling. And you may make new friends! I know that I did while working on this project.

VI. Version Control Choice and Reflection

For this project we chose to use Github for our Version Control, relying on Github Desktop to ease the process of pushing and pulling commits. We mainly chose to do this as it was familiar to the tech people on the team and we considered it a tried and true version control method. Unfortunately, we found out that it wasn't the best option around the alpha build time.

Unreal Engine 5 does not interface well with Github, as when an Unreal file is updated in engine, the binary files are not rebuilt. This is a crucial step to make projects work in Unreal and the biggest issue we ran into was our graybox/level file breaking. One person would edit the file and attempt to push to main and there would be a merge conflict that unreal would try to automatically resolve, which led to several issues.

In another project, I would love to look into Perforce, or even Microsoft Azure DevOps for a free option.